bixplot python documentation

04 May 2026

`bixplot`	*Boxplot version suited for bimodal and multimodal data, combining density, box, and rug elements with automatic cluster detection*

Description

Draws a bixplot for one or more numeric variables. A bixplot extends the violin plot and boxplot by automatically testing each variable for unimodality (via Hartigan's dip test) and, when multimodality is detected, fitting a constrained k-medoids clustering to identify and separately display the modes. Each variable is rendered as a filled density body, a box-and-whisker summary, and a rug of individual data values. The rug can optionally be colored by an external numeric or factor variable. Input can be provided as a DataFrame, array, list of arrays, Series, or dictionary.

For each variable, `bixplot` proceeds as follows. Hartigan's dip test is applied to test for unimodality; if the p-value exceeds `diplevel`, or if the sample is too small ($n < 2 \cdot \texttt{minN}$), the variable is treated as unimodal ($k = 1$). Otherwise, constrained k-medoids clustering is fitted for $k = 2, \ldots, \texttt{kmax}$ clusters, and the best k is selected based on the highest mean silhouette width. If no $k > 1$ yields a positive mean silhouette width, the variable is treated as unimodal. When `split = "True"` and `hue` has exactly 2 levels, densities are split symmetrically around center.

Usage

```
bixplot(data = None,
        x = None,
        y = None,
        orient = 'v',
        group_order = None,
        width = 0.8,
        diplevel = 0.01,
        minN = 15,
        clusMinN = 3,
        kmax = 5,
        maxit = 500,
        stand = False,
        verbose = False,
        hue = None,
```

```
hue_color = None,
hue_order = None,
split = True,
hue_legend = True,
legend_position = None,
mode_color = None,
mode_legend = False,
mode_color_by_suffix = True,
rug_hue = None,
rug_color = None,
rug_legend = False,
jitter = False,
rug_linewidths = 1,
showrug = True,
rug_colorbarthickness = 30,
rug_colorbarheight = 1,
rug_colorbarposition = None,
rug_length = 0.12,
rug_outer_color = None,
rug_outer_linewidths = 1.5,
box_color = 'black',
showbox = True,
box_linewidth = 1,
box_width = 1,
density_color = 'mode',
showdensity = True,
density_alpha = 0.5,
density_borderlinewidth = 1,
density_norm = 'width',
kernel = 'gaussian',
bandwidth = 'scott',
cut = 3,
cutmin = None,
cutmax = None,
bigN = 500,
undo_constrainkmax = False,
random_state = None,
ax = None)
```

Parameters

`data` : DataFrame, Series, dict, array, or list of arrays

Input dataset for plotting.

- DataFrame (long-form): provide column names in `x` and `y`.
- DataFrame (wide-form): each column is a distribution, converted internally to long-form.

- Series: passed directly as `x` or `y`.
- dict: keys are variable names, values are arrays/lists of observations.
- array or list of arrays: treated as distributions (wide-form).

If both `x` and `y` are None, `data` is interpreted as wide-form.
Otherwise, `data` is expected to be long-form and `x`, `y` specify the relevant columns.

`x, y` : str or None
Column names for x and y axes. If both omitted, data is interpreted as wide-form (i.e., a dataframe where each column is one variable (distribution you want to plot) and each row is one observation). Otherwise it is expected to be long-form.

`orient` : {'v', 'h'}, default='v'
Plot orientation: 'v' for vertical, 'h' for horizontal.

`group_order` : list, optional
Ordering of grouping axis

`width` : float, default=0.8
Width of the body

`diplevel` : float, default = 0.01
the level of Hartigan's dip test for unimodality of a variable. Only when this test rejects unimodality, that is, its p-value is at most diplevel, do we search for clusters. Defaults to 0.01. Setting it higher can lead to more clusters, and setting it lower to fewer clusters.

`minN` : int, default = 15
the minimum number of observations required per potential cluster. The maximum number of clusters searched is limited to n/minN, where n is the number of non-missing values. If $n < 2 * \texttt{minN}$, clustering is not attempted to avoid spurious clusters from small samples.

`clusMinN` : int, default = 3
The clustering is constrained so that any cluster must contain at least clusMinN unique values. When a variable has fewer values than this, only points are drawn.

`kmax` : int, default=5
Maximum number of clusters to test. It will be truncated to 7 internally. If NULL, it is set to min(floor(n/minN), 7).
When setting kmax=1 all variables are considered as single clusters, making the display resemble a violin plot.

`maxit` : int, default=500
Maximum number of iterations in the constrained clustering loop.

`verbose` : bool, defualt = False
If True, will print intermediate steps

`hue` : str, optional
Categorical variable for additional grouping. Supports > 2 categories (forces `split=False`).

`hue_color` : str | list | dict, optional
Defining the color of each hue levels. Accepts seaborn palette name, list of colors (will be recycled as needed), or dict mapping.

`hue_order` : list, optional
Explicit ordering of hue categories. Missing categories will be appended alphabetically.

`split` : bool, default=True
If True and hue has exactly 2 levels, densities are split symmetrically around center.

`hue_legend` : bool, default=True
If True, adds a legend for hue levels mapping the colour to the hue levels.

`legend_position` : str | (float, float), optional
Location of combined legend (modes, hue, rug). Accepts matplotlib legend locations (e.g. `'upper right'`) or `bbox_to_anchor` coordinates.

`mode_color` : str | list | dict, optional
Colors for modality clusters. Accepts seaborn palette, list of colours (will be recycled as needed), or dict mapping.

`mode_legend` : bool, default=False
If True, adds legend for detected modalities (clusters).

`mode_color_by_suffix` : bool, default=False
If True, assign colors based only on the modality suffix (e.g. `"_0"`, `"_1"`), so that the same clustering suffix across different groups shares the same color. If False, each full modalityID (group + suffix) gets its own color.

`rug_hue` : str or None, default=None
Column name in data providing the values that determine rug colors.

- If None → no rug mapping performed.
- If numeric → values mapped to continuous colormap.
- If categorical → values mapped to discrete palette or dict.

`rug_color`
When rug is the column name of categorical data: str | list | dict | None
When rug is the column name of numerical data: str | None
When rug is None: str
Default=None

Color specification for rug values:

- str: Single color (e.g. `"black"`). All rug ticks use the same color.
- str (palette name): Name of seaborn colormap (e.g. `"viridis"`, `"Set2"`).
- list: Explicit list of colors (categorical only).
- dict: Explicit mapping from category → color.

- None: Defaults to `"dark:salmon_r"` for numeric, `"Set2"` for categorical.

`rug_legend` : bool, default=False
If True, adds legend for rug values. If numeric, displayed as a colorbar.

`jitter` : bool, default=False
Add random jitter to rug positions to reduce overlap.

`rug_linewidths` : float, default=1
Line width of rug ticks.

`showrug` : bool, default=True
Whether to show rug plot. Forced to True when a rug column is provided.

`rug_colorbarthickness` : int, default=30
Aspect ratio of rug colorbar (larger → thinner bar).

`rug_colorbarheight` : float, default=1
Relative height of rug colorbar (1 = full axis).

`rug_colorbarposition` : [x, y, w, h], optional
Manual position of rug colorbar in figure coordinates.

`rug_length` : int
Size of the rug.

`rug_outer_color` : color or None, default=None
Specifies the color(s) of the part of the rug lines outside the body. If None, the same color as inside the body is used.

`rug_outer_linewidths` : float, default=1.5
Width of the rug lines outside the body.

`box_color` : str or {`'mode'`, `'hue'`}, default=`'black'`
Color mapping for boxplots. Can follow mode colors, hue colors, or fixed color

`showbox` : bool, default=True
Whether to display boxplots.

`box_linewidth` : float, default=1
Line width for boxplot elements.

`box_width` : float, default=1
Width of boxplots.

`density_color` : str | {`'mode'`, `'hue'`}, default=`'mode'`
Color mapping for densities. Can follow mode colors, hue colors, or fixed string.

`showdensity` : bool, default=True
Whether to display density estimates.

`density_alpha` : float, default=0.5
Transparency of density fill.

`density_borderlinewidth` : float, default=1
Line width of density border.

`density_norm` : `'area'` | `'count'` | `'width'`
Normalization method for densities.

- If area, each violin will have the same area.
- If count, the width will be proportional to the number of observations.
- If width, each violin will have the same width.

`kernel` `'gaussian'`, `'tophat'`, `'epanechnikov'`, `'exponential'`, `'linear'`, `'cosine'`, default='gaussian'
The kernel for use by klearn.neighbors.KernelDensity()

`bandwidth` : float or `'scott'`, `'silverman'`
Defines kernel bandwidth. If bandwidth is a string, one of the estimation methods is implemented.

`cut` : int, default=3
Controls how far the kernel density estimate extends beyond the minimum and maximum observed data. If set to 0, the density is clipped to the range of the data. Denoting a variable by y, the density is constructed over the interval

$$\min(y) - \texttt{cut} \cdot bw \quad \text{to} \quad \max(y) + \texttt{cut} \cdot bw,$$

where bw is the numeric bandwidth.

`cutmin` : int, default=None
If specified, the densities of all variables and modes cannot go below the value `cutmin`.

`cutmax` : int, default=None
If specified, the densities of all variables and modes cannot exceed the value `cutmax`.

`random_state` : int, optional
Random seed for reproducibility.

`bigN` : int, default=500
When a variable has more than `bigN` non-missing values, `bigN` values are sampled without replacement to reduce computation time.

`undo_constrainkmax` : bool, default=False
If True, prevents truncation of `kmax` to 5 internally.

`ax` : matplotlib Axes, optional
Axis on which to draw.

Returns

The function draws the bixplot and returns a tuple containing summary statistics (as a `dict`) and the input dataset augmented with cluster assignment labels (`modalityID`, as a `DataFrame`)):

dict A dictionary containing clustering and dip test results for each group (e.g. each level of `x` or combination of `x` and `hue`), including:

- `kmax`: maximum number of clusters considered.
- `dip_test_statistic`: value of Hartigan's dip statistic.
- `dip_test_p_value`: associated p-value for the dip test.
- `best_k`: selected number of clusters (when multimodality is detected).
- `silhouette_score`: mean silhouette width for the selected clustering.

`DataFrame` A copy of the input data with an additional column:

- `modalityID`: cluster assignment label (e.g. `"Biscoe_0"`), indicating the detected mode. Missing values are preserved as `NaN`.

Author(s)

C.M. Montalcini

References

Montalcini, C., Rousseuw, P.J. (2025). The bixplot: A variation on the boxplot suited for bimodal data, *doi:10.48550/arXiv.2510.09276* (open access).

Examples

```
# Load dataset
import seaborn as sns
df_penguins = sns.load_dataset("penguins")

# Bixplot by island with custom styling
res = bixplot(data=df_penguins, x='island', y='bill_length_mm',
              group_order=['Torgersen', 'Biscoe', 'Dream'],
              rug_length=0.20,
              rug_linewidths=0.4,
              box_width=2.5,
              box_color='black',
              box_linewidth=1.4)

# Extract original dataframe with an additional "modalityID" column
df_penguis_withmodes = res[1]

# Extract statistical results per group, here extracted for the 'Dream' island group
print(res[0]['Dream']['dip_test_statistic'],
      res[0]['Dream']['dip_test_p_value'],
      res[0]['Dream']['silhouette_score'])

For more examples, see \url{https://github.com/camontal/bixplot/tree/main/examples}.
```